TikZ Diagram Generator

Powered by IBM Granite & watsonx.ai

An AI-powered, research-assisted, multi-agent platform transforming natural-language descriptions into publication-quality LaTeX/TikZ diagrams.

Domain: Computer Science / AI / Technical Diagramming

Presenter: **Susnata Maiti**

github.com/Susnata-Maiti/TikZ-Diagram-Generator

IBM UNIVERSITY ENGAGEMENT | PROJECT SUBMISSION

Student & Project Information

PROJECT OVERVIEW

TikZ Diagram Generator

Domain: Computer Science / AI / Technical Diagramming

STUDENT DETAILS

Name: Susnata Maiti

Email: maitisusnata135531@gmail.com

Mobile: 8240506045

github.com/Susnata-Maiti/TikZ-Diagram-Generator



Problem Statement

⚠ The Challenge

- **Manual Complexity:** Drafting UML, ERDs, and circuits in TikZ is tedious.
- **Steep Learning Curve:** Existing tools demand expert LaTeX syntax fluency.
- **No Automation:** Plain English rarely compiles into valid diagrams.
- **Strict Notations:** 17+ diagram categories require distinct symbol standards.

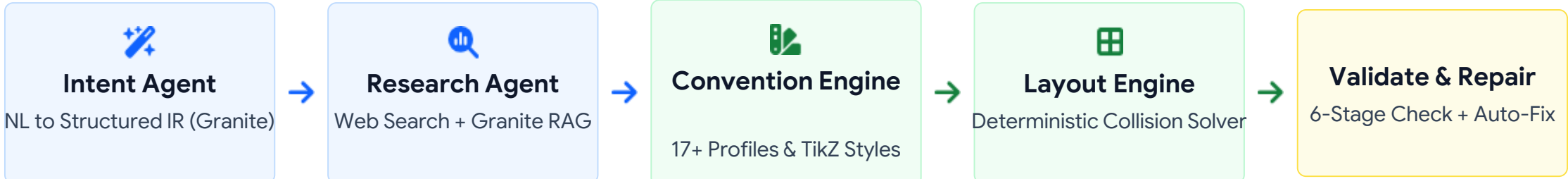
◎ The Objective

- **Multimodal Input:** Accept plain English, specs, or hand-drawn sketches.
- **Standards Compliance:** Support 17+ categories including UML, ERD, and circuits.
- **Validated Output:** Auto-repair and compile directly to PDF and PNG.
- **Agentic Intelligence:** Use IBM Granite for reasoning, repair, and RAG.

⚡ **Core Gap: No unified platform converts natural language into standards-compliant TikZ across 17+ diagram types.**

Proposed Solution

Multi-agent AI platform powered by IBM watsonx.ai and Granite converting plain English into compiled PDF & PNG diagrams.



System Outputs

- **Compiled Graphics:** Publication PDF & 220 DPI PNG preview
- **Source & Schema:** Raw TikZ/LaTeX code & structured IR JSON
- **Audit Artifacts:** 6-stage validation report & RAG citations

Key Capabilities

- **Multimodal & Broad:** 17+ categories & vision sketch-to-diagram
- **Resilient Pipeline:** Up to 3 autonomous Granite repair cycles
- **Interactive State:** Full undo/redo history & RAG enrichment

Technology Used

IBM watsonx.ai

Hosts Granite and Llama models with enterprise governance and retry fallback.

IBM Granite Models

Uses **granite-4-h-small** for IR/repair and **llama-4-maverick** for sketch vision.

FastAPI + Next.js

Python 3.11 async REST backend paired with React live preview and sketch upload.

pdflatex / TeX Live

Local compilation with texlive.net fallback producing PDF and 220 DPI PNGs.

Pydantic IR Schema

Central DiagramIR single source of truth for nodes, edges, groups, and geometry.

DuckDuckGo / SerpAPI

Live web RAG pipeline enriching prompts with real technical specifications.

Backend Components Used

1. GraniteClient
Wraps ModelInference for chat, JSON, and vision with 3-retry back-off.

2. IntentAgent
Parses natural language and refinement commands into DiagramIR JSON.

3. ReasoningAgent
Audits IR for UML members, circuit topology, and sequence lifelines.

4. ConventionRegistry
Defines shapes, TikZ libraries, styles, and routing for 17+ diagram types.

5. LayoutEngine
Runs topological sort, bounding boxes, and 10-step collision resolution.

6. TikZGenerator
Emits standalone LaTeX with UML splits, pgfonlayer groups, and anchors.

7. ValidationPipeline
Executes 6 checks from semantics to compilation plus 3 Granite repairs.

8. ResearchManager
Combines DuckDuckGo/SerpAPI search with Granite technical synthesis.

9. HistoryService
Maintains session undo/redo stacks with optional Redis persistence.

IBM UNIVERSITY ENGAGEMENT | TIKZ DIAGRAM GENERATOR

Multi-Agent Workflow

STAGE 1: AGENTIC INPUT & REASONING PIPELINE

Prompt / Sketch
User Input

→

ResearchAgent
RAG Context

→

IntentAgent
Builds IR JSON

→

ReasoningAgent
Logic Audit

→

Conventions
17+ Rulesets

STAGE 2: DETERMINISTIC LAYOUT, VALIDATION & COMPILATION

LayoutEngine
Exact Geometry

→

TikZGenerator
LaTeX Code

→

Validation
6-Stage + Repair

→

LaTeXCompiler
pdflatex / Cloud

→

REST Response
PDF • PNG • TikZ

Central DiagramIR (Pydantic): id • category • nodes • edges • groups • geometry • validation • research_sources

Granite NEVER outputs coordinates — all geometry is deterministic.

Supported Diagram Types

17+

Diagram Types

Dedicated ConventionProfile per category with strict notation rules.

UML Diagrams

Class Diagram • Use Case Diagram • Sequence Diagram • Activity Diagram • State Diagram

Flow & Process

Flowchart • Data Flow (DFD) • BPMN • ERD (Chen / Crow's Foot) • Resource Allocation Graph

Engineering & AI

Architecture Diagram • Neural Network • ML Pipeline • Electrical Circuit • Digital Logic Circuit

Specialized Notations

Quantum Circuit • Finite State Machine (FSM) / Decision Tree • General Graph / Network

IBM UNIVERSITY ENGAGEMENT | TIKZ DIAGRAM GENERATOR

Architecture Blueprint

CLIENT LAYER

Frontend (Next.js)

- pages/index.js
- useDiagramState.js
- lib/api.js & globals.css

REST ↔

API GATEWAY

Backend (FastAPI)

- POST /generate & /refine
- POST /upload-sketch
- GET /diagram/{id} • /undo

→

ORCHESTRATION

Core Services

- **DiagramService:** End-to-end flow
- **HistoryService:** Undo/redo stack

→

WATSONX.AI

Granite Agents

- Intent & Reasoning Agents
- ResearchAgent (Web RAG)
- SketchInterpreter (Vision)

Engines & Registry: LayoutEngine • ConventionRegistry • TikZGenerator • ValidationPipeline • LaTeXCompiler

Central Schema: DiagramIR (Pydantic) — Single source of truth passed across all layers

Infrastructure: Docker Worker | Vercel (Frontend) | texlive.net (Fallback Compiler) | Redis (Session History)

Role of Agentic AI in the Solution

IBM Granite agents perceive context, reason independently, and collaborate for standards compliance.

Intent Parsing

Maps free-form text into structured DiagramIR categories, nodes, and edges.

RAG Enrichment

Retrieves live web data and synthesizes real-world component lists.

Logical Review

Validates UML members, circuit topology, and lifelines before layout.

Self-Repair Loop

Diagnoses compiler error logs and fixes TikZ across 3 autonomous retries.

Vision-Based Sketch Interpretation (llama-4-maverick)

SketchInterpreter sends hand-drawn sketches to watsonx.ai vision models to extract node shapes, labels, and edge connections directly into a unified DiagramIR schema.

Token Usage Across Pipeline Stages

Pipeline Stage	Token Consumption	Input / Output Operation
Intent Parsing	**~1,200 – 2,500 tokens**	Natural language prompt to structured DiagramIR JSON
Technical Reasoning	**~800 – 1,500 tokens**	IR logical validation and topology correction
Research Agent (Optional)	**~2,000 – 4,500 tokens**	Live web snippets to synthesized technical architecture
Repair Loop (Per Retry)	**~1,500 – 3,000 tokens**	Compiler error log + TikZ to self-repaired TikZ (max 3x)
Vision / Sketch	**~500 – 1,200 tokens**	Hand-drawn sketch image to node/edge JSON via watsonx.ai

Standard Run (No Research, 1 Repair): ~5,000 – 9,000 tokens

Full RAG + Max Repairs: ~15,000 tokens (**ibm/granite-4-h-small**)

Novelty and Uniqueness

17+ Standards Profiles

Dedicated ConventionProfile shapes, libraries, and routing per diagram type.

Deterministic Geometry

Coordinates are never hallucinated; collision detection prevents overlaps.

Autonomous Self-Repair

pdflatex error logs trigger up to 3 autonomous Granite self-healing loops.

RAG Research Enrichment

Live web retrieval grounds diagrams in accurate domain architecture.

Sketch-to-Diagram Vision

Hand-drawn sketches convert into compiled LaTeX via llama-4-maverick.

IR-Driven Undo / Redo

Refinements mutate DiagramIR directly for guaranteed structural consistency.

GitHub Repository & Project Stats

https://github.com/Susnata-Maiti/TikZ-Diagram-Generator Branch: master

Repository Contents

- **backend/:** FastAPI endpoints, Granite agents, compiler, validation, tests
- **frontend/:** Next.js pages, custom React hooks, REST client, styles
- **worker/:** Production Dockerfile for containerized TeX Live deployment
- **Config & Docs:** .env.example, README.md, vercel.json, pyproject.toml

Repository Stats

- **54 Committed Files:** Full-stack implementation
- **Languages:** Python, JavaScript, Dockerfile
- **Security:** .gitignore protects secrets & .venv
- **Test Suite:** 6 pytest files (offline mock-ready)

Future Scope

1. CircuiTikZ Integration

Replace built-in IEC library with IEEE-compliant analog and electrical circuit symbols.

2. quantikz / yquant Support

Add multi-qubit gates (Toffoli, controlled-U), barrier lines, and quantum measurements.

3. Async Queue (Celery + Redis)

Run generation in background tasks for concurrent sessions and live progress tracking.

4. Multi-Language Output (Mermaid & PlantUML)

Emit Mermaid.js and PlantUML directly from the format-agnostic DiagramIR schema.

5. Combined Sequence Fragments & BPMN Swimlanes

Extend layered coordinate geometry to support UML alt/opt/loop blocks and swimlanes.

Certificates Earned — IBM SkillsBuild / IBM Bob

Completed: 23 Oct 2026 | ALM-COURSE_4071307



IBM SkillsBuild | Lab: Troubleshoot Your Code Using IBM Bob | Recipient: **Susnata Maiti**

IBM UNIVERSITY ENGAGEMENT | AICTE IBM OCTOBER INTERNSHIP

Thank You

TikZ Diagram Generator — Powered by IBM Granite & watsonx.ai

Susnata Maiti